VisionEngine

See the UI like a user — computer vision plus LLM vision for analysis and navigation.

Summary

VisionEngine is a decoupled Go toolkit that combines classic computer vision with LLM-based vision to analyze user interfaces, detect UI elements and visual issues, and build navigation graphs of app screen transitions — with pluggable multi-provider vision backends and OpenCV gated behind a build tag.

Short description

A reusable Go module for UI analysis and navigation-graph construction. It offers an analyzer layer (UI elements, screen diffs, visual issues), a navigation graph with BFS pathfinding and DOT/JSON/Mermaid export, and LLM-vision adapters for GPT-4o, Claude, Gemini, Qwen-VL, and more.

Long description

Most UI test automation is effectively blind. It reaches for accessibility trees and DOM selectors — a machine's idea of an interface — and misses everything a human actually experiences: whether a button is visibly rendered, whether the layout broke, whether the screen it landed on is the one it expected. VisionEngine closes that gap by giving automation genuine perception, the ability to look at a UI and reason about it the way a person would. It is organized into four cooperating layers that build from raw pixels up to whole-app understanding. The **Analyzer** defines the stable contract — interfaces (`Analyzer`, `VideoProcessor`) and value types (`UIElement`, `ScreenAnalysis`, `ScreenDiff`, `Rect`, `Size`, `TextRegion`, `VisualIssue`, `ScreenIdentity`, `Action`, `KeyFrame`) with a `StubAnalyzer` reference implementation — so consumers can detect elements, diff screens, and surface visual issues against a contract that won't shift under them. The **NavigationGraph** lifts the view from a single screen to the whole

application, modelling it as a directed graph of screen transitions with BFS pathfinding and three export back-ends (DOT, JSON, Mermaid), so automation can not only see a screen but plan a route to any other — and it ships stress, automation, integration, and security test suites to prove it. The **LLM Vision** layer adds modern multimodal reasoning: a `VisionProvider` interface with adapters for OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica, and Ollama, composed via a `FallbackChain` so a failing, rate-limited, or weak provider degrades gracefully to the next instead of taking the run down with it. A **Configuration** layer handles env-var loading and validation with every user-facing error string routed through the `i18n.Translator` seam.

The decision that makes all of this actually adoptable is that the heavy native dependency is optional. OpenCV bindings are build-tag-gated behind `-tags vision`, and the default build ships stubs — so the entire module compiles, tests, and runs on any Go 1.25+ host with no OpenCV toolchain in sight, and only pulls in the native stack when a consumer explicitly opts into it. That is what lets VisionEngine drop into a plain CI runner without a bespoke image. Fully decoupled per the constitution (CONST-051(B)), it is incorporated by consumers — notably HelixQA — as an equal-codebase submodule, giving evidence-driven UI testing a real pair of eyes.

Why we built it

UI test automation that relies only on accessibility trees or selectors misses what the user actually sees. VisionEngine adds real visual understanding — element detection, screen diffing, and LLM-vision reasoning — plus a navigable map of app screens, so automation can both perceive and route through a UI.

Why it’s a game-changer

It puts two normally-incompatible approaches — fast, deterministic classic computer vision and flexible, semantic LLM vision — behind a single interface with a fallback chain, so a consumer gets the precision of one and the reasoning of the other without choosing. And by keeping OpenCV strictly optional, it removes the usual tax on that power: any Go project can gain real UI perception without dragging a native vision toolchain into its build.

What’s innovative

- Dual perception: classic CV (OpenCV/GoCV) plus multi-provider LLM vision with a fallback chain.
- Navigation graph with BFS pathfinding and DOT/JSON/Mermaid export.
- Build-tag-gated OpenCV so the module stays buildable/testable without native deps.
- Fully decoupled, i18n-seamed, equal-codebase submodule (used by HelixQA).

Challenges & solutions

- **Heavy native dependency friction:** solved with `-tags vision` gating and default stubs so CI/hosts without OpenCV still build and test.
- **Vision-provider unreliability:** solved with a `VisionProvider` interface and `FallbackChain` composer.
- **Mapping complex app flows:** solved with a directed navigation graph plus BFS pathfinding and multi-format export.
- **Coupling:** solved via CONST-051(B) decoupling and the i18n translator seam.

Tech stack (why + how)

- **Go (1.25+)** — module core and all four layers.
- **GoCV / OpenCV** — classic computer vision, build-tag-gated.
- **LLM vision providers (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — multimodal UI reasoning via adapters.
- **Graph algorithms (BFS)** — navigation pathfinding.
- **DOT / JSON / Mermaid exporters** — navigation-graph visualization.
- **i18n Translator** — decoupled user-facing strings.